An Open-Source Plug-and-Play Testbed for Reproducible SDN Controller Performance Benchmarking

Tamerlan Abdullayev
Department of Computer Science
University of Cincinnati
Cincinnati, OH, USA
abdulltn@mail.uc.edu

Giovani Abuaitah
Department of Computer Science
University of Cincinnati
Cincinnati, OH, USA
abuaitgi@ucmail.uc.edu

***Abstract*—Reproducible and rigorous benchmarking of Software-Defined Networking (SDN) controllers remains an open challenge. Existing tools either rely on synthetic traffic that fails to capture realistic controller behavior, or consist of ad-hoc manual experiments that lack statistical validation and can be challenging to reuse across studies. We present an open-source, plug-and-play SDN benchmarking testbed that combines Mininet, iperf3, and an instrumented Ryu controller application into a fully automated, seed-controlled experimentation pipeline. The testbed supports configurable topologies and parameters, records per-event controller processing latency (Packet-In to FlowMod dispatch), and produces statistically validated results through repeated trials with inline aggregation. To demonstrate the testbed, we execute 400 controlled experiments across two topologies (star and fat-tree), five host counts, and four flow rates with 10 repetitions per configuration. The demonstration confirms that the testbed successfully captures topology-dependent controller behavior and uncovers performance differences that are invisible to existing benchmarking approaches. The testbed and all experiment scripts are publicly available at: https://github.com/PLACEHOLDER/sdn-testbed.**

***Index Terms*—SDN, benchmarking, testbed, reproducibility, Mininet, OpenFlow, controller performance, iperf3, open-source.**

I. Introduction

Software-defined networking (SDN) [1] decouples the control plane from the data plane, placing forwarding intelligence in a software controller. The OpenFlow protocol [1] provides a standardized southbound interface between controllers and compatible switches, enabling programmatic network control. As SDN deployments scale [2], rigorous and reproducible benchmarking of controller performance becomes increasingly important for SDN research, controller selection, scalability assessment, and capacity planning.

The Reproducibility Gap. Despite the availability of several benchmarking tools, the SDN research community lacks an open-source, end-to-end testbed that is both statistically rigorous and reusable across studies. Synthetic tools such as CBench [3], OFLOPS [4], and WCBench [5] inject artificial Packet-In bursts to measure controller throughput but generate no real TCP flows, perform no MAC learning, and may not capture the topology-driven event dynamics that characterize realistic deployments. The IETF's RFC 8456 [6] defines a methodology for SDN controller benchmarking—recommending varying topologies, varying network sizes, and repeated statistical trials—yet to the best of our knowledge, open-source implementations of that methodology remain limited.

Recent academic studies that do use realistic emulation tools such as Mininet and iPerf [7]–[9] rely on single, manually executed runs between one host pair with no statistical repetitions, no automated parameter sweeps, and no controller instrumentation beyond end-to-end RTT. As a result, their findings can be challenging to verify, extend, or reuse without reconstructing the experimental setup independently.

Our Contribution. We present an open-source, plug-and-play SDN benchmarking testbed that addresses these gaps. The testbed makes the following contributions:

- **Automated, reproducible pipeline.** A seed-controlled, fully automated shell and Python pipeline orchestrates controller startup, network emulation, traffic generation, per-run aggregation, and cleanup. Any experiment can be exactly replicated from its seed without manual intervention.
- **Per-event controller instrumentation.** The testbed instruments the Ryu controller [10] at the event level, recording wall-clock timestamps from Packet-In receipt to FlowMod dispatch for every control-plane event. This enables separation of median (p50) and tail (p95) latency, a distinction invisible to RTT-based measurements used in prior work.
- **Realistic concurrent traffic.** Traffic is generated by iperf3 [11] with concurrent, overlapping TCP flows between randomly selected host pairs, creating realistic simultaneous Packet-In events that stress the controller in ways that synthetic single-flow tests may not.
- **Plug-and-play extensibility.** Topologies and traffic parameters are configurable via command-line arguments. New topologies are added by implementing a single Python class. The architecture is designed for straight-

forward substitution of alternative controllers.

- **Statistical validation.** The pipeline supports configurable repetitions per configuration with distinct seeds, enabling standard deviation computation and confidence evaluation across trials.

To the best of our knowledge, this is among the first open-source frameworks that aligns with the IETF RFC 8456 benchmarking methodology recommendations while supporting realistic concurrent TCP traffic, per-event controller instrumentation, and automated multi-dimensional parameter sweeps.

To validate the testbed, we execute 400 controlled experiments across star and fat-tree topologies, five host counts (4–64), and four flow rates (25–200 flows/sec) using 10 repetitions per configuration. These experiments serve as a demonstration of the testbed's measurement capabilities, illustrating its ability to surface statistically validated, topology-dependent performance differences that prior benchmarking approaches may not detect.

The testbed and all scripts are openly available at: https://github.com/PLACEHOLDER/sdn-testbed.

The remainder of this paper is organized as follows. Section II surveys related tools and their limitations. Section III describes the testbed architecture and design. Section IV presents the validation experiments and results. Section V discusses findings and future extensions. Section VI concludes.

II. Related Work and Limitations of Existing Tools

A. Synthetic Benchmarking Tools

The dominant tool for SDN controller benchmarking is CBench [3], which emulates switches sending Packet-In messages at the controller and counts FlowMod responses per second. While widely cited, CBench uses no real topology, generates no TCP flows, and performs no MAC learning, and therefore may not fully capture the topology-driven event dynamics that arise in realistic deployments. Zhu et al. [12] compared 34 controllers using CBench, PktBlaster, and OFNet, explicitly noting that *none of the tools available can measure all performance statistics* and that benchmarking tool selection significantly affects outcomes.

OFLOPS [4] provides an open framework for switch-level evaluation but focuses on the data plane rather than controller performance. WCBench [5] wraps CBench with automation and statistics collection but is locked to a specific OpenDaylight release and appears no longer actively maintained. HCprobe [13] extends CBench with reliability and security tests but retains the same synthetic traffic model. All of these tools share a fundamental limitation: they measure controller responsiveness to artificial stimulus, not behavior under realistic concurrent TCP flows.

B. IETF Benchmarking Standards and Gaps

The IETF Benchmarking Methodology Working Group (BMWG) has published several relevant standards. RFC 8456 [6] defines methodology for SDN controller performance benchmarking, recommending tests across multiple topologies and varying network sizes with statistically validated repeated trials. RFC 5180 [14] further recommends that throughout benchmarking tests, *"the Device Under Test (DUT) can be monitored for relevant resource (processor, memory, etc.) utilization,"* recognizing that resource monitoring provides beneficial insight into traffic processing demands. Our testbed follows both of these recommendations: it conducts systematic multi-topology sweeps with repeated trials as suggested by RFC 8456, and records per-second CPU and memory utilization of the controller process as suggested by RFC 5180.

These standards, however, address methodology only and have not yet prescribed open-source tooling or implementations. To the best of our knowledge, publicly available open-source testbeds that implement these recommendations for SDN controller benchmarking in an automated, reusable form remain limited.

TABLE I
Comparison of SDN Benchmarking Approaches

Feature	CBench/ OFLOPS	Prior Studies	Our Testbed
Real TCP flows	✗	✓	✓
Automated pipeline	✗	✗	✓
Statistical reps.	✗	✗	✓
Param. sweep	✗	✗	✓
Per-event latency	✗	✗	✓
p50/p95 separation	✗	✗	✓
CPU/memory monitoring	✗	✗	✓
Open-source code	✓	✗	✓
Plug-and-play topo.	✗	✗	✓

✗ = absent; ✓ = present.

C. Academic Studies Using Mininet and iPerf

Several recent studies use Mininet [15] and iPerf for more realistic evaluation. Koulouras et al. [7] compared ONOS, Ryu, Floodlight, and OpenDaylight in fat-tree topologies using Mininet and iPerf, but ran single manual experiments between one host pair per scenario with no repetitions and no controller instrumentation. Čabarkapa et al. [8] evaluated Ryu and ONOS in a custom fat-tree using Mininet and iPerf, manually running each test three times to check consistency, with no parameter sweep and no automation. Ali [9] compared four controllers across five topologies sending 50 packets per scenario—again manually, with a single run per configuration and no statistical analysis.

These studies generally do not release code, which can make independent replication challenging without reconstructing the full setup. Table I summarizes the key methodological gaps addressed by our testbed.

III. Testbed Architecture and Design

A. Overview

Fig. 1 illustrates the testbed architecture. Four components interact under the control of an automated orchestration layer: (1) the *instrumented controller*, (2) the *network emulator*, (3) the *traffic generator*, and (4) the *system monitor*. A master shell

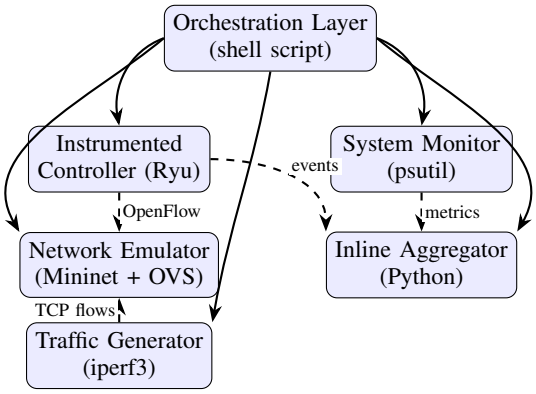


Fig. 1. Testbed architecture. The orchestration layer coordinates all components; the aggregator consolidates per-run outputs.

script coordinates their startup, execution, and teardown, and invokes an inline aggregation pass that appends one summary row per run to a persistent CSV file before deleting large raw outputs to manage storage.

B. Instrumented Controller

The controller component is a custom Ryu [10] application that extends standard MAC-learning L2 switching with three instrumentation layers. First, every Packet-In event is timestamped on arrival using `time.perf_counter()`; the processing time to FlowMod dispatch is recorded in microsecond resolution to a per-run CSV file (`flow_events.csv`). This enables direct measurement of controller processing latency, independent of network RTT. Reporting both the median (p50) and 95th-percentile (p95) of these per-event times is essential: as Xiong et al. [16] and the IETF BMWG note, mean latency does not reflect tail behavior in event-driven systems under bursty load, and it is precisely the tail events that reveal controller saturation and topology-driven anomalies. Second, aggregate Packet-In and FlowMod counts are written to a separate CSV (`controller_metrics.csv`) at one-second intervals via a background greenthread (`hub.spawn`), ensuring the logger does not block the main event loop. Third, the output directory is injected via the `OUTDIR` environment variable, allowing the orchestrator to direct each run's output to an isolated directory without modifying controller code.

C. Network Emulator and Topology Layer

Mininet [15] provides network emulation using Open vSwitch [17] with TCLink. Topologies are defined as Python classes inheriting from Mininet's `Topo` base class. Currently, the testbed ships three built-in topologies: star, fat-tree, and mesh. Adding a new topology requires implementing a single `build(self, n)` method and registering it with a one-line entry in the topology factory. This design allows researchers to plug in any Mininet- compatible topology without modifying the rest of the pipeline.

D. Traffic Generator

iperf3 [11] servers are started on each host at unique port numbers (base port + host index) and allowed one second to bind before traffic begins. The traffic generator maintains a precise inter-flow interval of $1/\text{FPS}$ seconds using wall-clock timing with sleep compensation to absorb scheduling jitter. For each flow, source and destination hosts are selected uniformly at random without replacement using a deterministic seed that is unique per (topology, host count, flow rate, repetition) tuple. Flows run concurrently as subprocesses, creating overlapping TCP connections that generate simultaneous Packet-In events and stress the controller in a way that single-flow benchmarks may not.

Require: topology T, host count N, flow rate F, duration D, seed s, output directory *out*
1: Clean residual Mininet state (`mn -c`)
2: Set `OUTDIR` $\leftarrow$ *out*
3: Launch Ryu controller $\rightarrow$ background PID p_c
4: Launch system monitor (psutil, PID p_c) $\rightarrow$ background
5: **sleep**(1) {Allow controller to bind}
6: Launch Mininet(T, N) + iperf3 traffic gen (seed s, FPS F, dur D)
7: **wait** for all iperf3 clients to complete
8: Kill system monitor; kill Ryu controller
9: **sleep**(1) {Allow file handles to flush}
10: Run `aggregate_results.py` $\rightarrow$ append row to `summary.csv`
11: **if** aggregation succeeded **then**
12: Delete `iperf_json/`, `ryu_stdout.log`, `mininet_iperf.log`
13: Retain `flow_events.csv`, `controller_metrics.csv`, `system_metrics.csv`, `traffic_meta.json`
14: **else**
15: Retain all raw files for manual inspection
16: **end if**

Fig. 2. Testbed orchestration pipeline (`run_experiment.sh`).

E. System Monitor

A companion Python process uses `psutil` [18] to record the controller process's CPU utilization, resident memory (RSS), and thread count at one-second intervals for the duration of each run. CPU percentage is primed with an initial non-blocking call before sampling begins, following psutil best practice. The monitor runs as a separate OS process to avoid any GIL contention with the controller.

F. Orchestration and Reproducibility

The master shell script (`run_experiment.sh`) accepts topology, host count, flow rate, run duration, flow duration, and seed as command-line arguments. Algorithm 2 shows the full orchestration logic. Seeds are computed deterministically as seed $= 1000 + N \cdot 10 + \text{FPS} + \text{rep}$, guaranteeing that any individual run can be exactly reproduced from its parameter tuple alone. The matrix scripts iterate over all (topology, N, FPS, repetition) combinations and invoke `run_experiment.sh` unattended for the full factorial sweep.

G. Metrics and Aggregation

After each run, the inline aggregator (`aggregate_results.py`) reads the three output

TABLE II
VALIDATION EXPERIMENT PARAMETERS

Parameter	Values
Topologies	Star, Fat-Tree
Host count (N)	4, 8, 16, 32, 64
Flow rate (FPS)	25, 50, 100, 200 flows/sec
Flow duration	2 seconds
Run duration	20 seconds
Repetitions	10 (distinct seeds per config)
Total runs	400 (200 per topology)
Controller	Ryu v4.34 (OpenFlow 1.3)
Emulator	Mininet + OVSSwitch + TCLink
Traffic tool	iperf3 (TCP, JSON output)

CSVs and the metadata JSON, computes the following metrics, and appends one row to a master summary CSV:

- **Throughput (Gbps):** sum of `bits_per_second` across all iperf3 flows, converted from the per-flow JSON output.
- **Packet-In rate (pkts/sec):** mean of the per-second counts in `controller_metrics.csv`.
- **Processing latency p50 and p95 (ms):** 50th and 95th percentiles of `proc_time_ms` across all events in `flow_events.csv`.
- **CPU utilization (%):** mean CPU percent across all one-second samples in `system_metrics.csv`.
- **Memory (MB):** mean RSS across all samples.

IV. TESTBED VALIDATION: A CASE STUDY WITH RYU

A. Experimental Configuration

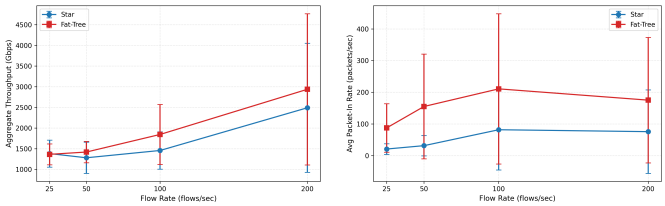
To validate the testbed and demonstrate its measurement capabilities, we run a full factorial experiment using the Ryu [10] controller (v4.34, OpenFlow 1.3) across two topologies, five host counts, and four flow rates. Table II summarizes the configuration. All experiments run on a single VirtualBox VM (Ubuntu 24.04, 16 vCPUs, 16 GB RAM). The full 400-run sweep was executed unattended using the matrix scripts.

B. Throughput and Packet-In Rate

Fig. 3 shows aggregate throughput and Packet-In rate versus flow rate. Fat-tree achieves higher throughput at most configurations (2938 vs. 2491 Gbps at 200 fps), demonstrating that the testbed correctly captures topology-driven throughput differences. The fat-tree topology generates substantially more Packet-In events because each aggregation switch contributes independently to the controller event stream, producing up to $5\times$ more events than the star topology at the same flow rate. Both topologies show a slight Packet-In decline at 200 fps as flow table entries are reused—a behavioral nuance that only per-event instrumentation can reveal.

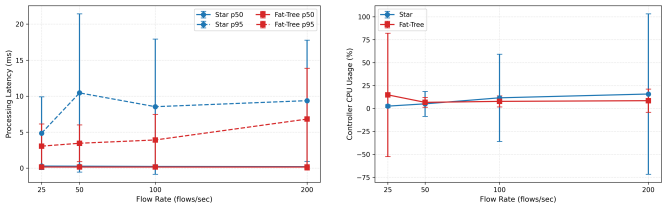
C. Per-Event Processing Latency

Fig. 4 presents the most significant demonstration of testbed capability: per-event processing latency at p50 and p95 across topologies and flow rates. The p50 latency is near zero for both topologies, confirming that median-latency measurements—the only metric available in prior work—would show no



(a) Aggregate throughput vs. flow rate (b) Packet-In rate vs. flow rate

Fig. 3. Throughput and Packet-In rate vs. flow rate. Error bars show one standard deviation across 10 repetitions.



(a) Processing latency (p50 and p95) vs. flow rate (b) CPU usage vs. flow rate

Fig. 4. Processing latency and CPU usage vs. flow rate. p50 is near zero for both topologies; p95 reveals large topology-dependent differences invisible to RTT-based measurements. Error bars show one standard deviation.

meaningful difference between topologies. However, the p95 latency tells a strikingly different story: star topology p95 peaks at 10.44 ms at 50 fps before declining, while fat-tree p95 grows gradually from 3.04 ms to 6.80 ms, consistent with queuing theory predictions [16].

This p50/p95 divergence is directly attributable to the testbed's per-event instrumentation. No existing open-source benchmarking tool measures Packet-In-to-FlowMod latency at event granularity; prior studies using ping and iPerf RTT report only end-to-end delay, which conflates network latency with controller processing latency.

D. Scalability with Host Count

Fig. 5 and Table III reveal a critical scalability divergence as host count increases. Star topology p95 latency grows $24.4\times$ from 0.73 ms at $N{=}4$ to 17.81 ms at $N{=}64$, whereas fat-tree p95 grows only $10.9\times$ (0.68 to 7.44 ms), remaining $2.4\times$ lower at maximum scale. The star's p95 plateaus near 17 ms between $N{=}32$ and $N{=}64$, suggesting saturation of a burst-driven phenomenon.

V. DISCUSSION

A. What the Testbed Revealed

The validation experiments surface five findings that demonstrate the testbed's measurement capabilities. **(F1)** Topology and host count drive p95 latency far more than flow rate in the tested range—a finding only accessible through systematic multi-dimensional parameter sweeps, a capability that prior open-source tools have not yet offered. **(F2)** The p50/p95 latency gap demonstrates that tail behavior is driven by a small fraction of events and is invisible to median-based or RTT-based measurements. **(F3)** Star topology p95 latency grows near-quadratically with host count ($0.73 \rightarrow 17.81$ ms, $24.4\times$)

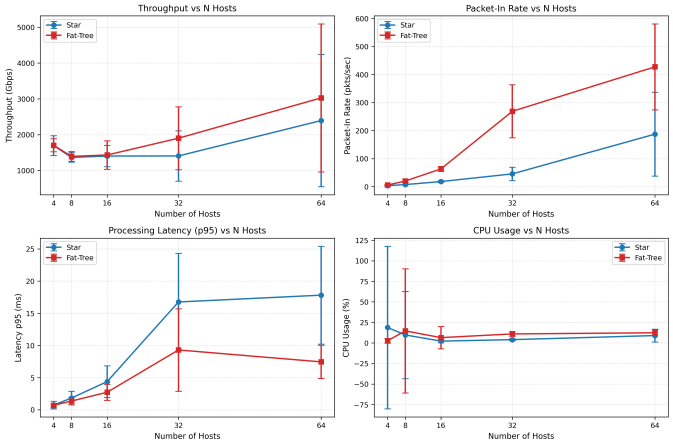


Fig. 5. Scalability: throughput, Packet-In rate, p95 latency, and CPU vs. host count. Star p95 plateaus at $\sim$17 ms beyond N=32; fat-tree remains lower across all scales.

TABLE III
MEAN PERFORMANCE METRICS BY TOPOLOGY AND HOST COUNT

Topo.	N	Tput (Gbps)	Pkt-In (pkt/s)	p95 Lat (ms)	CPU (%)
Fat-Tree	4	1703.93	5.88	0.68	2.70
Fat-Tree	8	1393.19	20.03	1.35	14.66
Fat-Tree	16	1431.88	63.42	2.72	6.47
Fat-Tree	32	1901.14	268.79	9.29	10.98
Fat-Tree	64	3026.39	427.36	7.44	12.36
Star	4	1696.51	3.69	0.73	18.78
Star	8	1363.72	7.40	1.82	9.74
Star	16	1402.91	18.16	4.37	2.21
Star	32	1405.18	45.58	16.75	4.01
Star	64	2393.71	187.36	17.81	8.99

but saturates between N=32 and N=64, consistent with a MAC learning burst of $N(N-1)/2$ unique host pairs temporarily saturating the controller's event queue. **(F4)** Fat-tree topology partitions MAC learning across aggregation switches, bounding burst intensity to $\leq$6 host pairs per switch regardless of total network size, which explains its 2.4$\times$ lower tail latency at scale. **(F5)** CPU utilization remains below 16% for both topologies, confirming that the observed latency anomalies are event-queue driven rather than compute-resource limited.

Critically, the MAC burst phenomenon in (F3) and (F4) was discovered only because the testbed instruments individual Packet-In events: a study using only aggregate RTT measurements would observe a higher average latency for star topologies but could not distinguish burst-phase from steady-state behavior, nor identify the $N(N-1)/2$ scaling relationship.

B. Limitations of the Demonstration

The current demonstration uses a single-threaded controller (Ryu). Ryu's Python Global Interpreter Lock (GIL) prevents true parallel event processing [13], making it a tractable but architecturally constrained baseline. The testbed is designed for straightforward extension to multi-threaded or distributed controllers such as ONOS [19] or OpenDaylight.

Emulation fidelity is a secondary limitation: Mininet with OVSSwitch in VirtualBox introduces timing noise not present in physical hardware deployments. Absolute throughput values reflect the emulated environment; relative comparisons across topologies and configurations remain valid.

C. Testbed Roadmap

The current release is a foundation; our planned development focuses on upgrading the testbed itself along three stages. **Containerization:** The most immediate priority is replacing Mininet's network namespaces with per-host Docker containers, giving each emulated node a fully isolated filesystem, process space, and network stack. Running the controller, hosts, and traffic generator as containers makes the topology behave closer to a real deployment and makes the entire testbed portable—any researcher could bring it up with a single `docker compose up` command on any machine, with no manual dependency installation. **Configurable infrastructure:** Building on the containerized foundation, we plan to expose link-level parameters—bandwidth, latency, and loss rate—as first-class configuration alongside topology and traffic parameters, enabling researchers to model a wider range of network conditions without modifying any code. **Web-based interface:** The longer-term goal is a browser-based frontend through which a researcher can select a topology, set all experiment parameters, launch a run, and retrieve result plots—removing the command-line requirement entirely and making the testbed accessible to the broader SDN research community.

VI. CONCLUSION

We presented an open-source, plug-and-play SDN controller benchmarking testbed that addresses three core limitations of the current landscape: the reliance on synthetic traffic in established tools such as CBench and OFLOPS, the lack of statistical rigor and automation in academic emulation studies, and the absence of any open-source implementation of the IETF's RFC 8456 benchmarking methodology.

The testbed contributes a seed-controlled automated pipeline, per-event controller instrumentation that separates p50 from p95 processing latency, concurrent realistic TCP traffic via iperf3, and a plug-and-play topology and parameter system. A 400-run validation study using Ryu across star and fat-tree topologies confirms that the testbed successfully captures statistically validated, topology-dependent controller behavior—including a MAC learning burst anomaly that is undetectable by prior benchmarking approaches.

The testbed is publicly available at https://github.com/PLACEHOLDER/sdn-testbed. Planned development will progressively upgrade the tool itself—toward containerized per-host isolation, richer link-level configuration, and a web-based interface—with the goal of making rigorous SDN controller benchmarking accessible to the broader research community.

REFERENCES

[1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[2] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[3] Stanford University, "CBench: Controller benchmarking tool," https://github.com/mininet/oflops, 2012.

[4] C. Rotsos, N. Sarrar, S. Uhlig, R. Sherwood, and A. W. Moore, "OFLOPS: An open framework for OpenFlow switch evaluation," in *Proceedings of the 13th International Conference on Passive and Active Measurement (PAM)*, 2012.

[5] D. Farrell, "WCBench: CBench, wrapped in stuff that makes it useful," https://github.com/dfarrell07/wcbench, 2015.

[6] V. Bhuvaneswaran, A. Basil, M. Tassinari, V. Manral, and S. Banks, "Benchmarking methodology for software-defined networking (SDN) controller performance," Internet Engineering Task Force, RFC 8456, 2018.

[7] I. Koulouras, S. V. Margariti, I. Bobotsaris, E. Stergiou, and C. Stylios, "On the performance of SDN controllers in real world topologies," in *2022 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2022, pp. 143–148.

[8] D. Čabarkapa, D. Rančić, P. Pavlović, and M. Milićević, "Investigating the impact of tree-based network topology on the SDN controller performance," *Facta Universitatis, Series: Automatic Control and Robotics*, vol. 21, no. 1, pp. 25–35, 2022.

[9] U. Ali, "Performance comparison of SDN controllers in different network topologies," *Research Square (preprint)*, 2024, dOI: 10.21203/rs.3.rs-4826985/v1.

[10] NTT Software Innovation Center, "Ryu SDN framework," http://osrg.github.com/ryu, 2012.

[11] J. Dugan, S. Elliott, B. A. Mah, J. Poskanzer, and K. Prabhu, "iperf3: A TCP, UDP, and SCTP network bandwidth measurement tool," https://software.es.net/iperf/, 2014.

[12] L. Zhu, M. M. Karim, K. Sharif, X. Du, and M. Guizani, "SDN controllers: Benchmarking & performance evaluation," *arXiv preprint arXiv:1902.04491*, 2019.

[13] A. Shalimov, D. Zuikov, D. Zimarina, V. Pashkov, and R. Smeliansky, "Advanced study of SDN/OpenFlow controllers," in *Proceedings of the 9th Central & Eastern European Software Engineering Conference (CEE-SECR)*, 2013.

[14] C. Popoviciu, A. Hamza, G. V. de Velde, and D. Dugatkin, "IPv6 benchmarking methodology for network interconnect devices," Internet Engineering Task Force, RFC 5180, 2008, iETF Benchmarking Methodology Working Group.

[15] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*, 2010.

[16] B. Xiong, K. Yang, J. Zhao, W. Li, and K. Li, "Performance evaluation of OpenFlow-based software-defined networks based on queueing model," *Computer Networks*, vol. 102, pp. 172–185, 2016.

[17] Open vSwitch, "OVS: Production quality, multilayer open virtual switch," https://www.openvswitch.org/, 2026, accessed: 2026-06-08.

[18] G. Rodola, "psutil: Cross-platform lib for process and system monitoring in Python," https://github.com/giampaolo/psutil, 2026, accessed: 2026-06-12.

[19] P. Berde, M. Gerola, J. Hart, Y. Higuchi, M. Kobayashi, T. Koide, B. Lantz, B. O'Connor, P. Radoslavov, W. Snow, and G. Parulkar, "ONOS: Towards an open, distributed SDN OS," in *Proceedings of the Third Workshop on Hot Topics in Software Defined Networking (HotSDN)*, 2014.